

Sesión de Aprendizaje 3: Manejo de Formularios en PHP

Duración: 2 horas

Nivel: Intermedio

Objetivos:

- Comprender el manejo básico de formularios en PHP.
 - Utilizar las superglobales `$_GET` y `$_POST` para manejar datos.
 - Implementar validaciones de formularios y gestionar errores.
 - Aplicar medidas de seguridad en el procesamiento de formularios.
-

1. Introducción a los Formularios en PHP

- **Concepto:** Los formularios HTML permiten a los usuarios enviar datos al servidor.
 - **Uso de PHP:** PHP utiliza las superglobales `$_GET` y `$_POST` para gestionar los datos enviados por los formularios.
 - **Importancia de la Seguridad:** Es crucial validar y sanitizar la entrada del usuario para prevenir ataques como XSS (Cross-Site Scripting) y SQL Injection.
-

2. Crear un Formulario HTML Simple

Ejemplo de Formulario:

html

Copiar código

```
<html>
<body>
<form action="welcome.php" method="POST">
    Name: <input type="text" name="name"><br>
    E-mail: <input type="text" name="email"><br>
    <input type="submit">
</form>
</body>
</html>
```

- - **Explicación:**
 - **action:** Define el archivo que procesará el formulario.
 - **method:** Especifica el método de envío (**POST** o **GET**).
 - **Ejercicio Práctico:** Crear un formulario que incluya campos adicionales como "Teléfono" y "Dirección".
-

3. Manejo de Datos con PHP

Código de `welcome.php`:

php

Copiar código

```
<html>
<body>
Welcome <?php echo htmlspecialchars($_POST["name"]); ?><br>
Your email address is: <?php echo htmlspecialchars($_POST["email"]);
?>
</body>
</html>
```

- - **Explicación del Código:**
 - `htmlspecialchars()`: Previene la inyección de código HTML/JavaScript.
 - Mostrar los datos recibidos.
 - **Ejercicio Práctico:** Modificar `welcome.php` para mostrar todos los datos enviados.
-

4. Diferencias entre GET y POST

- **Teoría:**
 - **`$_GET`**: Datos en la URL, visible, límite de 2000 caracteres.
 - **`$_POST`**: Datos en el cuerpo de la solicitud, más seguro, sin límite estricto.

Ejemplo de Formulario con GET:

html

Copiar código

```
<form action="welcome_get.php" method="GET">
    Name: <input type="text" name="name"><br>
    E-mail: <input type="text" name="email"><br>
    <input type="submit">
</form>
```

-

Código de `welcome_get.php`:

php

Copiar código

```
<html>
<body>
Welcome <?php echo htmlspecialchars($_GET["name"]); ?><br>
Your email address is: <?php echo htmlspecialchars($_GET["email"]);
?>
```

```
</body>
</html>
```

- - **Ejercicio Práctico:** Comparar los resultados entre métodos GET y POST.
-

5. Seguridad en el Manejo de Formularios

- **Teoría de la Seguridad:**
 - Importancia de la validación de datos.
 - Riesgos asociados a la falta de validación.
- **Métodos de Sanitización:**
 - `htmlspecialchars()`
 - `filter_var()`

Ejemplo de Sanitización:

php

Copiar código

```
$name = htmlspecialchars($_POST["name"]);
$email = htmlspecialchars($_POST["email"]);
```

- - **Ejercicio Práctico:** Implementar medidas de seguridad en un formulario existente.
-

6. Validación de Formularios en PHP

- **Reglas de Validación:**
 - **Nombre:** Requerido, solo letras y espacios.
 - **Correo:** Requerido, debe ser una dirección válida.
 - **Sitio Web:** Opcional, debe ser una URL válida si se proporciona.
 - **Comentario:** Opcional.
 - **Género:** Requerido, debe seleccionarse uno.

Código de Validación:

php

Copiar código

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Validación de Nombre
    if (empty($_POST["name"])) {
        $nameErr = "Name is required";
    } else {
        $name = test_input($_POST["name"]);
        if (!preg_match("/^[a-zA-Z-' ]*$/", $name)) {
```

```

        $nameErr = "Only letters and white space allowed";
    }
}

// Validación de Correo
if (empty($_POST["email"])) {
    $emailErr = "Email is required";
} else {
    $email = test_input($_POST["email"]);
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $emailErr = "Invalid email format";
    }
}

// Validación de URL
if (!empty($_POST["website"])) {
    $website = test_input($_POST["website"]);
    if
(!preg_match("/\b(?:(:https?|ftp):\/\/|www\.)[-a-z0-9+&@#\/%?=_|!:\
,.;]*[-a-z0-9+&@#\/%=_|!:/i", $website)) {
        $websiteErr = "Invalid URL";
    }
}

// Validación de Género
if (empty($_POST["gender"])) {
    $genderErr = "Gender is required";
} else {
    $gender = test_input($_POST["gender"]);
}
}

```

•

Función `test_input()`:

php

Copiar código

```

function test_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}

```

```
}
```

-
- **Ejercicio Práctico:** Implementar validaciones adicionales según la lógica de negocio.

7. Mantenimiento de Valores en el Formulario

- **Teoría:**
 - Mantener valores permite una mejor experiencia del usuario después de un error de validación.

Ejemplo de Mantenimiento:

html

Copiar código

```
<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
    Name: <input type="text" name="name" value="<?php echo $name; ?>">
    <span class="error">* <?php echo $nameErr; ?></span>
    <br><br>
    E-mail: <input type="text" name="email" value="<?php echo $email; ?>">
    <span class="error">* <?php echo $emailErr; ?></span>
    <br><br>
    Website: <input type="text" name="website" value="<?php echo $website; ?>">
    <span class="error"><?php echo $websiteErr; ?></span>
    <br><br>
    Comment: <textarea name="comment" rows="5" cols="40"><?php echo $comment; ?></textarea>
    <br><br>
    Gender:
        <input type="radio" name="gender" <?php if (isset($gender) && $gender=="female") echo "checked";?> value="female">Female
        <input type="radio" name="gender" <?php if (isset($gender) && $gender=="male") echo "checked";?> value="male">Male
        <input type="radio" name="gender" <?php if (isset($gender) && $gender=="other") echo "checked";?> value="other">Other
        <span class="error">* <?php echo $genderErr; ?></span>
        <br><br>
        <input type="submit" name="submit" value="Submit">
```

</form>

- - **Ejercicio Práctico:** Asegurarse de que todos los campos se mantengan después de un error de validación.
-

8. Buenas Prácticas y Consideraciones Finales

- **Teoría:**
 - Mantener el código limpio y organizado.
 - Usar comentarios para facilitar la comprensión.
 - **Errores Comunes:**
 - No validar datos.
 - No sanitizar entradas.
 - No manejar errores adecuadamente.
 - **Recomendaciones:**
 - Usar frameworks que faciliten la gestión de formularios (como Laravel, Symfony).
 - Revisar regularmente las actualizaciones de PHP y sus mejores prácticas.
-

9. Conclusión y Preguntas

- Resumen de los temas cubiertos.
 - Espacio para preguntas y aclaraciones.
-

10. Recursos Adicionales

- **Documentación oficial de PHP:** [PHP Forms](#)
 - **Seguridad en PHP:** OWASP PHP Security
 - **Tutoriales en línea:** Listar algunos sitios recomendados.
-

Tarea Opcional

- Desarrollar un formulario que almacene los datos en una base de datos, aplicando todas las validaciones y medidas de seguridad vistas en clase.
- Considerar la implementación de un sistema de registro y autenticación básico.

SESIÓN EN AULA:

Problema 1: Mayor de Tres Números

Descripción: Crea un formulario que permita al usuario ingresar tres números y determinar cuál es el mayor.

Código HTML y PHP

html

Copiar código

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Mayor de Tres Números</title>
</head>
<body>
    <h1>Calcula el Mayor de Tres Números</h1>
    <form action="" method="POST">
        <input type="number" name="a" placeholder="Ingresa el primer
número" required>
        <input type="number" name="b" placeholder="Ingresa el
segundo número" required>
        <input type="number" name="c" placeholder="Ingresa el tercer
número" required>
        <input type="submit" value="Calcular">
    </form>

    <?php
    if ($_SERVER['REQUEST_METHOD'] == "POST") {
        $a = $_POST['a'];
        $b = $_POST['b'];
        $c = $_POST['c'];

        if ($a === $b && $b === $c) {
            echo "Todos los números son iguales.";
        } else {
            $mayor = max($a, $b, $c);
            echo "El mayor es: $mayor";
        }
    }
}
```

```
    ?>
</body>
</html>
```

Problema 2: Generación de Productos

Descripción: Desarrolla un sistema que genere un listado de productos basado en un número ingresado por el usuario.

Código HTML y PHP

html

Copiar código

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Generación de Productos</title>
</head>
<body>
    <h1>Generar Productos</h1>
    <form action="" method="POST">
        <input type="number" name="cantidad" placeholder="Cantidad
de productos" required>
        <input type="submit" value="Generar">
    </form>

    <?php
if ($_SERVER['REQUEST_METHOD'] == "POST") {
    $cantidad = $_POST['cantidad'];
    $productos = [];

    for ($i = 0; $i < $cantidad; $i++) {
        $productos[] = "Producto " . $i;
    }

    echo "<h2>Lista de Productos</h2>";
    echo "<table border='1'><tr><th>Producto</th></tr>";
    foreach ($productos as $producto) {
        echo "<tr><td>$producto</td></tr>";
    }
    echo "</table>";
```

```
    }  
    ?>  
</body>  
</html>
```

Problema 3: Gestión de Usuarios en la Base de Datos

Descripción: Crea un sistema CRUD básico para gestionar usuarios en una base de datos.

Código para la conexión a la base de datos (**conexion.php**)

php

Copiar código

```
<?php  
$servername = "localhost";  
$username = "root";  
$password = "";  
$dbname = "mi_base_datos";  
  
$conn = new mysqli($servername, $username, $password, $dbname);  
  
if ($conn->connect_error) {  
    die("Conexión fallida: " . $conn->connect_error);  
}  
?>
```

Código PHP para el CRUD de Usuarios

html

Copiar código

```
<!DOCTYPE html>  
<html lang="es">  
<head>  
    <meta charset="UTF-8">  
    <title>Gestión de Usuarios</title>  
</head>  
<body>  
    <h1>Gestión de Usuarios</h1>  
    <form method="POST">  
        <input type="text" name="nombre" placeholder="Nombre"  
required>
```

```

        <input type="email" name="email" placeholder="Email"
required>
        <button type="submit" name="agregar">Agregar
Usuario</button>
    </form>

    <?php
    include 'conexion.php';

        if (($_SERVER['REQUEST_METHOD'] == 'POST' &&
isset($_POST['agregar']))) {
            $nombre = $_POST['nombre'];
            $email = $_POST['email'];
            $conn->query("INSERT INTO usuarios (nombre, email) VALUES
('$nombre', '$email')");
        }

    $usuarios = $conn->query("SELECT * FROM usuarios");
    ?>

    <table border="1">
        <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Email</th>
            <th>Acciones</th>
        </tr>
        <?php while ($row = $usuarios->fetch_assoc()): ?>
        <tr>
            <td><?php echo $row['id']; ?></td>
            <td><?php echo $row['nombre']; ?></td>
            <td><?php echo $row['email']; ?></td>
            <td>
                <form method="POST" style="display:inline;">
                    <input type="hidden" name="id" value="<?php echo
$row['id']; ?>">
                    <button type="submit"
name="eliminar">Eliminar</button>
                </form>
            </td>
        </tr>
    </table>

```

```

        <?php endwhile; ?>
    </table>

    <?php
    if (isset($_POST['eliminar'])) {
        $id = $_POST['id'];
        $conn->query("DELETE FROM usuarios WHERE id='$id'");
    }

    $conn->close();
    ?>
</body>
</html>

```

Problema 4: Filtrado de Títulos

Descripción: Desarrolla una funcionalidad que permita filtrar títulos en la base de datos según criterios definidos por el usuario.

Código PHP para Filtrado

html

Copiar código

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Filtrar Títulos</title>
</head>
<body>
    <h1>Filtrar Títulos</h1>
    <form method="POST">
        <input type="text" name="filtro" placeholder="Ingrese
criterio de búsqueda" required>
        <button type="submit">Filtrar</button>
    </form>

    <?php
    include 'conexion.php';

    $filtro = '';
    if ($_SERVER['REQUEST_METHOD'] == 'POST') {

```

```

        $filtro = $_POST['filtro'];
    }

    $result = $conn->query("SELECT * FROM titulos WHERE titulo LIKE
'%$filtro%'");
    ?>

<table border="1">
    <tr>
        <th>ID</th>
        <th>Título</th>
    </tr>
    <?php while ($row = $result->fetch_assoc()): ?>
    <tr>
        <td><?php echo $row['id']; ?></td>
        <td><?php echo $row['titulo']; ?></td>
    </tr>
    <?php endwhile; ?>
</table>

<?php $conn->close(); ?>
</body>
</html>

```

Problema 5: Gestión de Asientos en un Cine

Descripción: Desarrolla un sistema que gestione la ocupación y disponibilidad de asientos en un cine.

Creación de la tabla de asientos (SQL)

sql

Copiar código

```

CREATE TABLE asientos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    fila INT NOT NULL,
    columna INT NOT NULL,
    ocupado BOOLEAN DEFAULT FALSE
);

```

Código PHP para la gestión de asientos

php

Copiar código

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Gestión de Asientos</title>
    <style>
        .ocupado { background-color: red; }
        .disponible { background-color: green; }
    </style>
</head>
<body>
    <h1>Gestión de Asientos en el Cine</h1>
    <table border="1">
        <?php
            include 'conexion.php';

            // Simulando que ya tenemos asientos en la base de datos
            for ($i = 1; $i <= 5; $i++) { // 5 filas
                echo "<tr>";
                for ($j = 1; $j <= 10; $j++) { // 10 columnas
                    // Consultar si el asiento está ocupado
                    $result = $conn->query("SELECT ocupado FROM asientos
WHERE fila=$i AND columna=$j");
                    $asiento = $result->fetch_assoc();
                    $ocupado = $asiento['ocupado'];

                    echo "<td class='" . ($ocupado ? "ocupado" :
"disponible") . "'>

                        <form method='POST'>
                            <input type='hidden' name='asiento'
value='$i$j'>
                                <button type='submit' name='reservar'>"
. ($ocupado ? "Ocupado" : "Reservar") . "</button>
                        </form>
                    </td>";

                    // Reservar el asiento si se presiona el botón
                    if (isset($_POST['reservar']) && $_POST['asiento']
== "$i$j" && !$ocupado) {
```

```
        $conn->query("UPDATE asientos SET ocupado=1
WHERE fila=$i AND columna=$j");
        echo
"<script>window.location.reload();</script>"; // Recargar la página
    }
}
    echo "</tr>";
}
?>
</table>
</body>
</html>
```

Notas Finales:

- Asegúrate de tener la base de datos configurada correctamente y de inicializar los datos de asientos en la tabla.
- Puedes ampliar cada uno de los